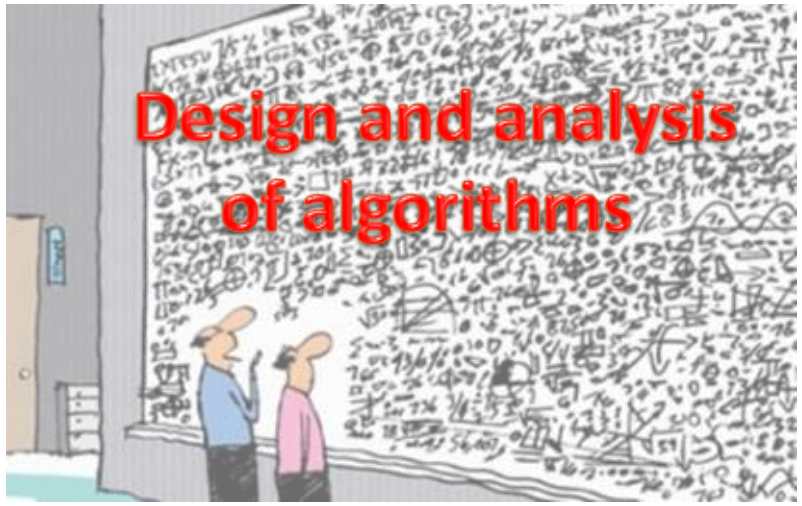


Analysis of graphs

- Strongly connected components

Objective

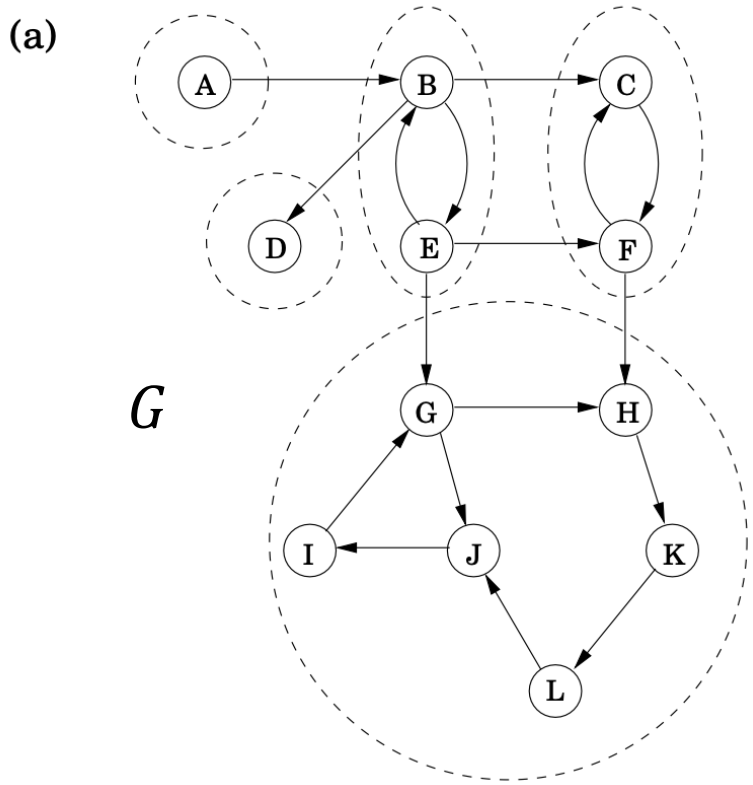
- Decompose a graph in sub-graphs where nodes are fully communicating.
- Many applications to random walks, etc.
- Kosaraju's algorithm.
- DPV Chapter 3.4.
- CLRS Ch 22.5.
- Play with animations (also for DFS and BFS) https://visualgo.net/en/dfsbfs.

Strongly connected components

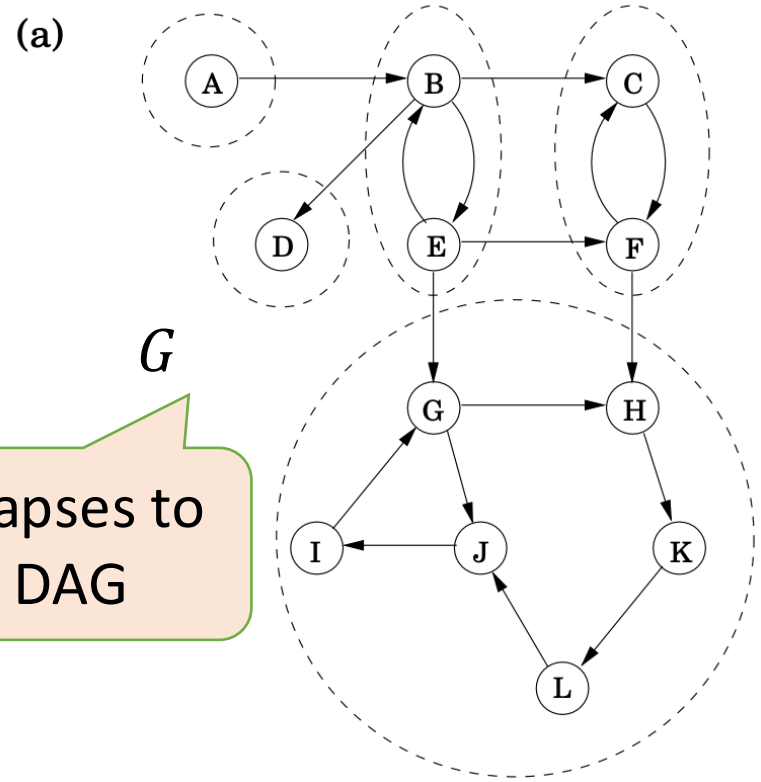
maximal: cannot add more vertices

- Strongly connected component: maximal set of vertices $C \subseteq V$ such that every pair of states *communicate.*
  - for every pair of vertices $u, v \in C$, we have both $u \rightsquigarrow v$ and $v \rightsquigarrow u$.

$u \rightsquigarrow v$: there is a path from u to v



The graph of SCCs is a DAG

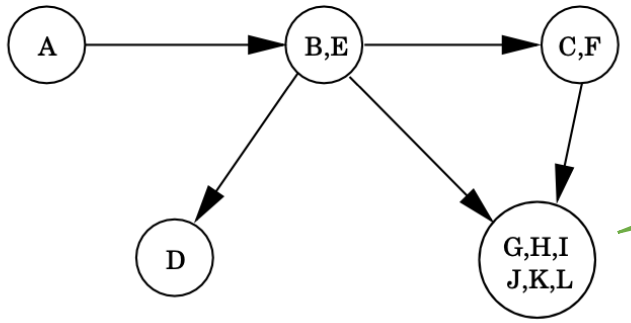


G

Collapses to a DAG

(b)

G^{SCC}: graph of SCCs



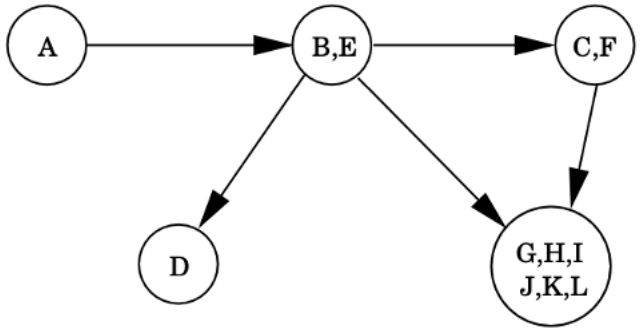
"sink" SCC: no outgoing edges in the DAG

topological order

$$A < BE < D < CF < GHIJKL$$

Property: Consider any topological order for the SCCs, e.g.,: $SCC_1 < \cdots < SCC_{k-1} < SCC_k$. Then, SCC_k is a sink (no outgoing edges). If we remove SCC_k, then SSC_{k-1} becomes a sink.

Properties of any DFS



$A < BE < D < CF < GHIJKL$

Property of DAGs:

finishing time

- If there is an edge $u \to v$, then $post(u) > post(v)$.

Property of any graph G: Given SCCs C, C',

- If there is an edge $C \to C'$, then $post(C) > post(C')$, where $post(C) =$ largest post time of nodes in C.

Corollary:

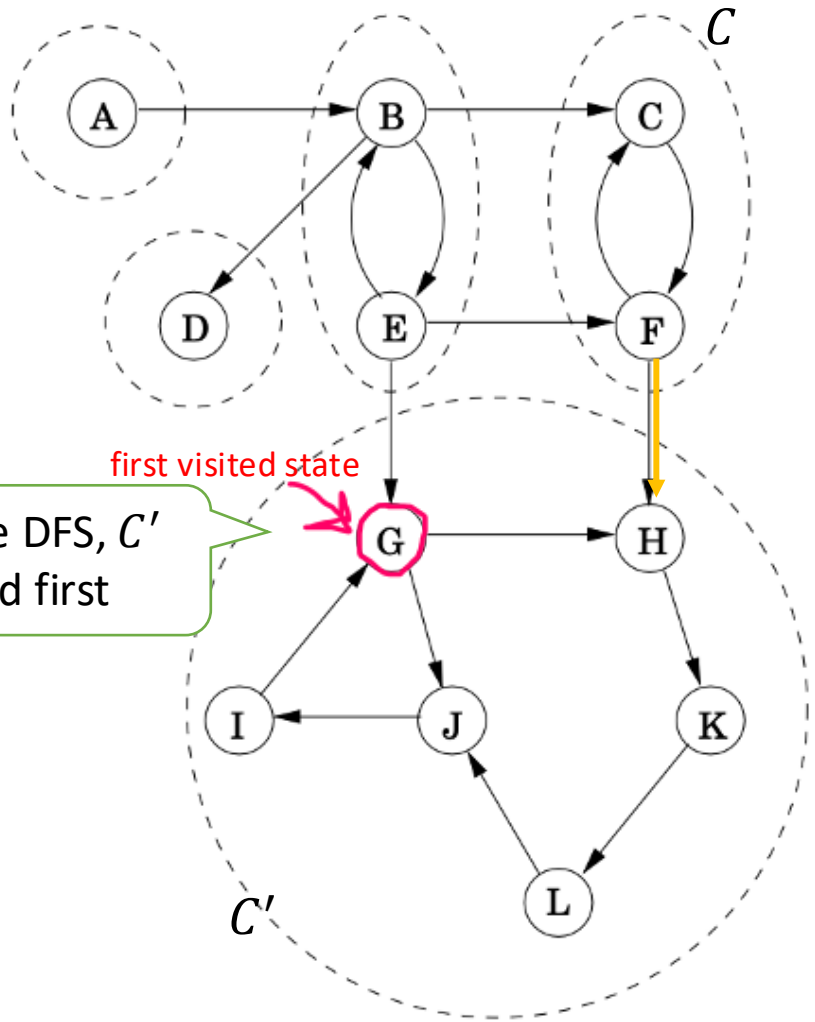
- We do a DFS on G, this defines an ordering $>_{DF}$ on V according to decreasing post times.

← highest post time

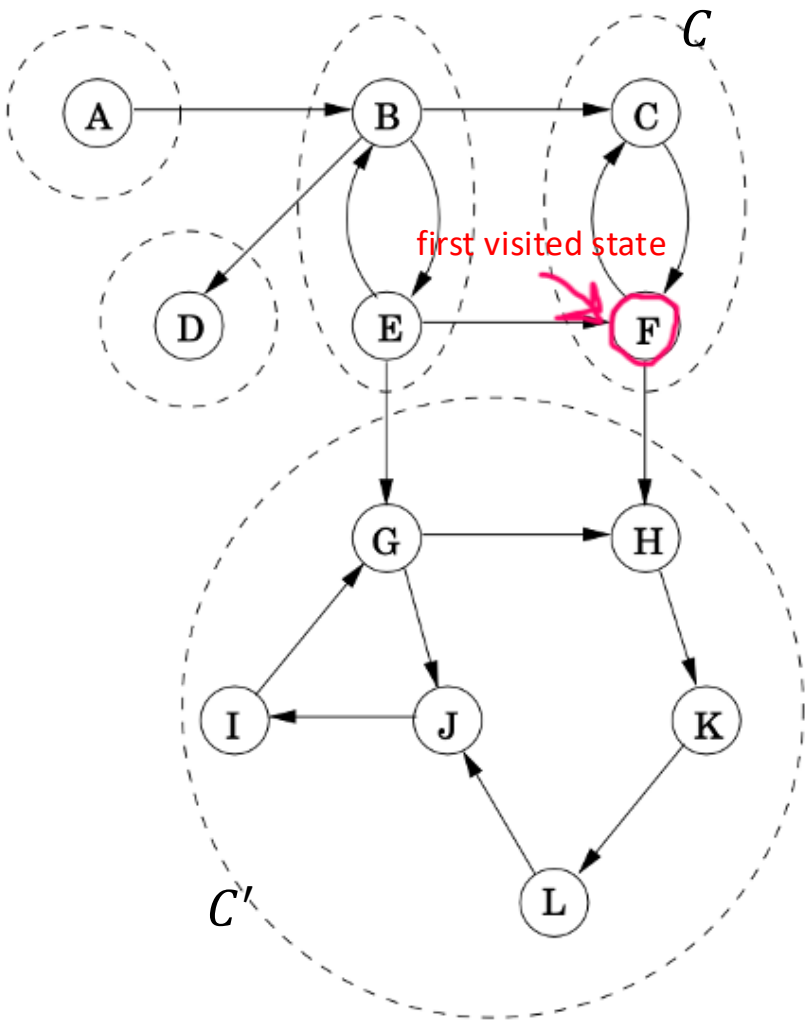
- If we order $u_1 > u_2 > u_3 > \cdots > u_n$ according to $>_{DF}$, then $u_1 \in SCC_1$ = source SCC.
- We consider the graph $G' = G - SCC_1$. if the remaining nodes are $u_1' > u_2' > u_3' > \cdots$ according to $>_{DF}$, then $u_1' \in SCC_2$ = source SCC of the SCC DAG of G'.

Proof: Apply the white path theorem. Consider two cases:

During the DFS, C' is visited first

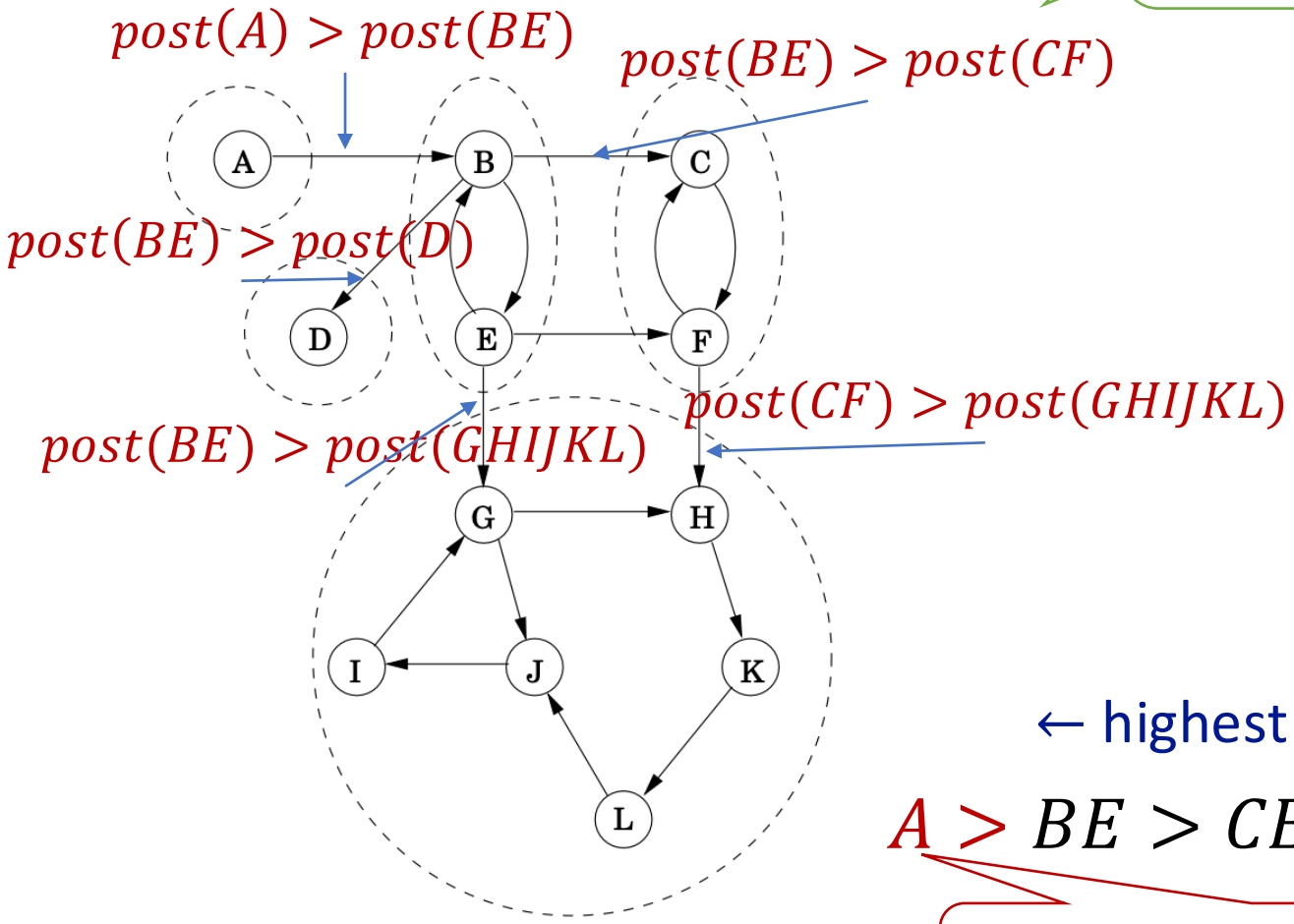


During the DFS, C is visited first



Example

Any DFS will create post times that satisfy these properties



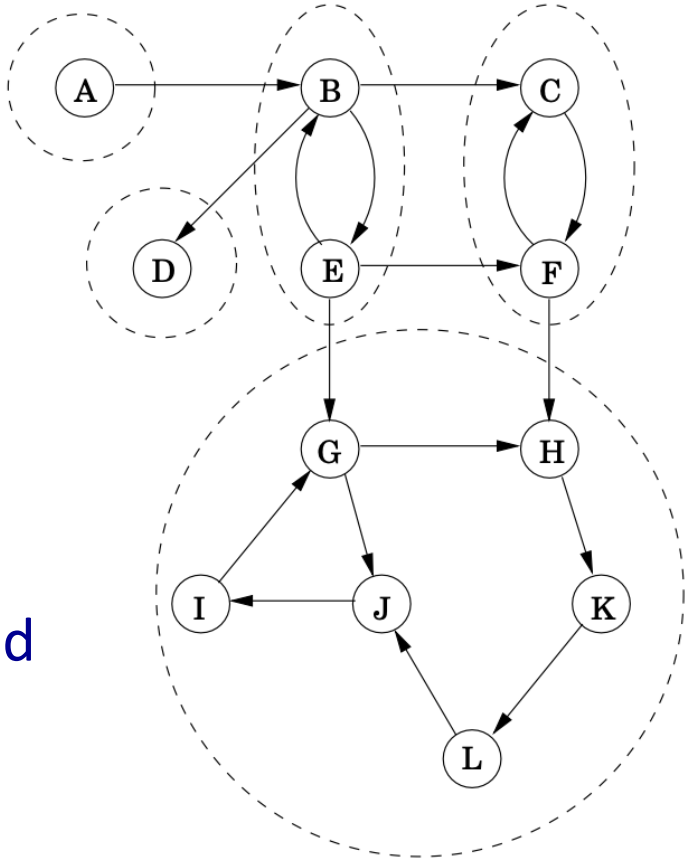
← highest post time

$$A > BE > CE > D > GHIJKL$$

A source SCC has the node with the largest finishing time of the DFS

Basic idea

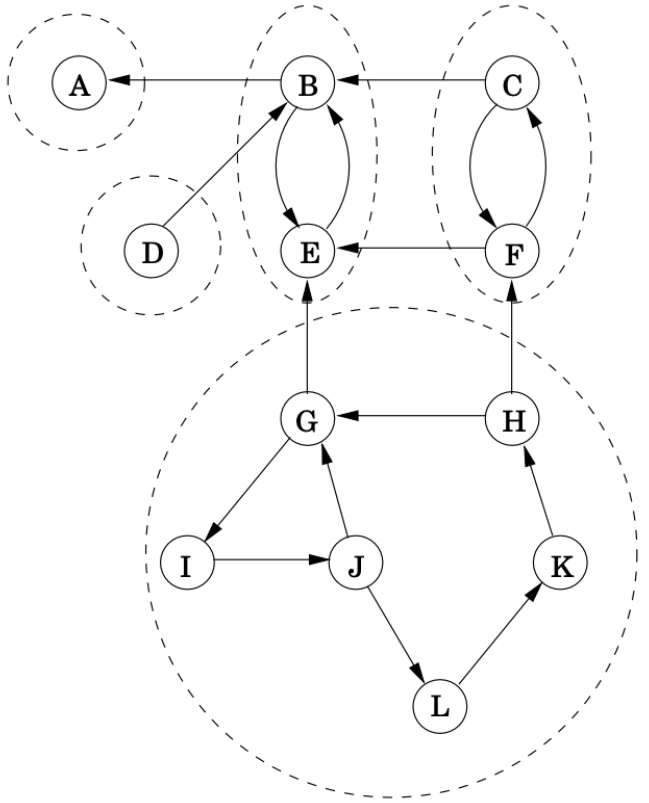
- ***Oracle-based*** algorithm 
- The oracle can always find a node in a sink SCC of G.
- **Algorithm**: Ask the oracle for a node u_1 in some sink SCC of G; run $DFS(G, u_1)$ to find the nodes of this SCC and extract them from G to get the new graph G'; repeat this on G' , etc, until the graph is empty.
- Example: Oracle gives node $J \in GHIJKL$, run DFS from J, find and take out $GHIJKL$ from the graph. Next, in the new graph the oracle gives us node $C \in CF$, we take out CF, etc.

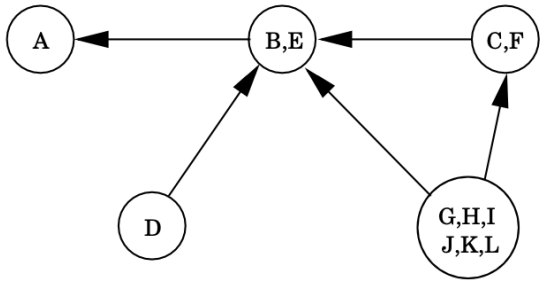


- *But we don't have an oracle to help us.* **Ideas how to get nodes in sink SCCs?** 

Basic idea

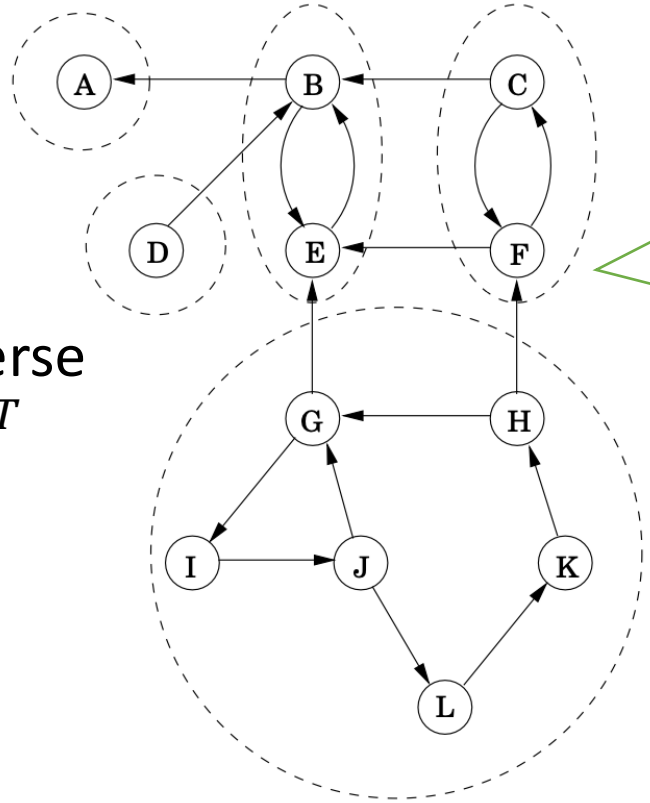
- **Instead of the oracle**: reverse the direction of the edges: sink SCCs become source SCCs!
  - Run a DFS to compute post values in the **reversed** graph. The highest post value is a node in a source SCC of the reversed graph ⇒ sink SCC of the original graph.
- **Algorithm idea**: Run DFS in the original graph to find the SCCs one by one by starting on nodes with the highest post value from the DFS on the *reversed graph*.



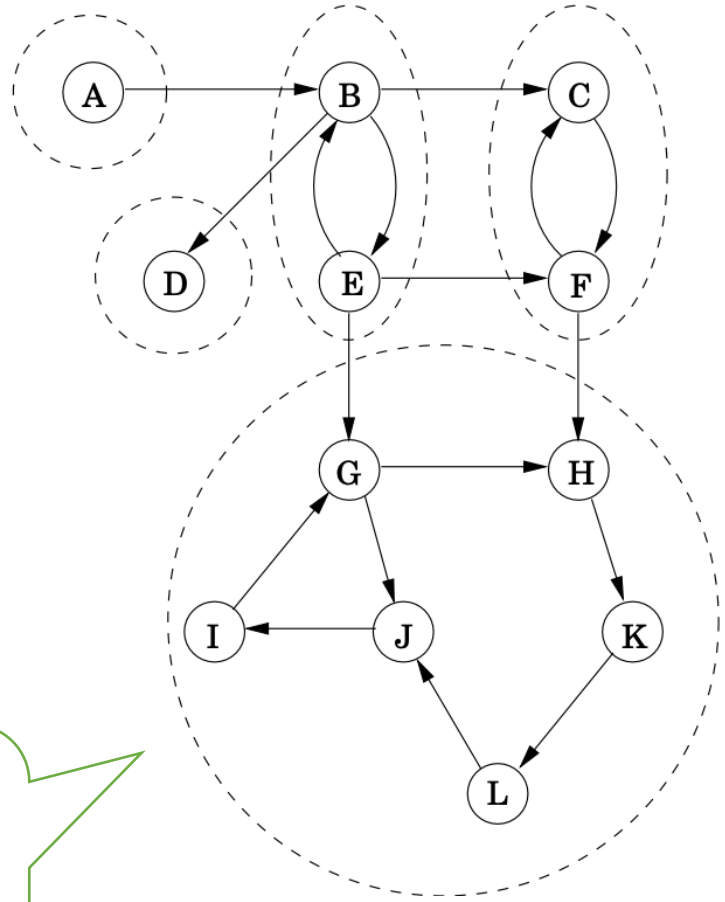


The reverse graph G^T

The reverse graph G^T



Step 1: do DFS on reversed graph to determine post values for nodes

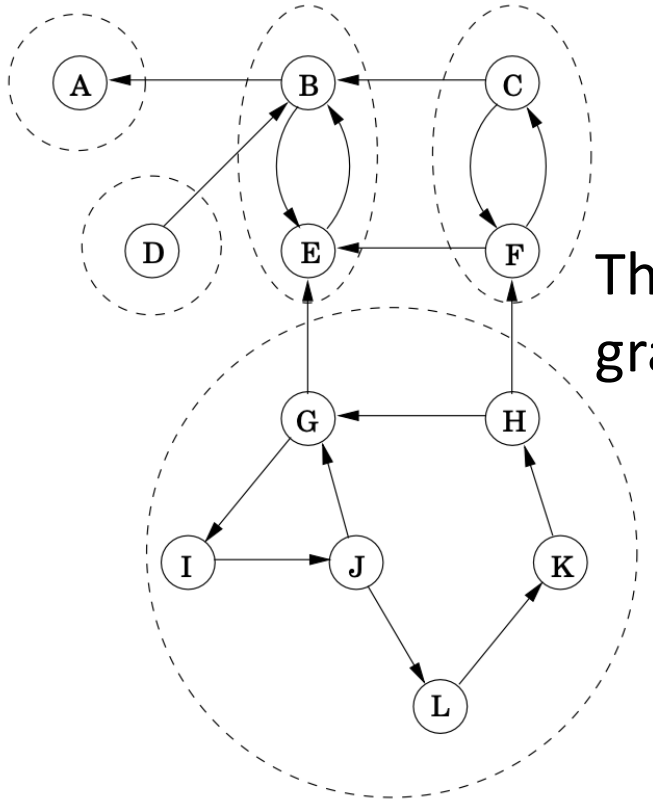


Step 2: Do DFS on original graph choosing starting nodes according to highest post values from step 1

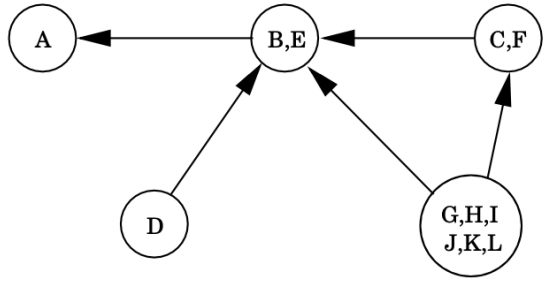
A “reverse” solution

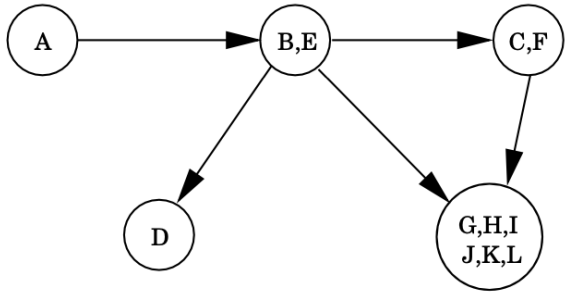
1) Do DFS on G and determine post values.
2) Reverse the graph and do a DFS on G^T by choosing nodes with priority according to their post values in 1).

- The DFS will visit first all nodes in SSC_1 and stop, then in SSC_2, etc. where $SSC_1, SSC_2, \ldots, SSC_k$ is a consistent topological order of the DAG of G.



The reverse graph G^T





The original graph G

Conclusions

- We discovered that using 2 DFS we can find the SCCs!
- Complexity: $O(|V| + |E|)$.